Command Prompt - jshell

```
Microsoft Windows [Version 10.0.22631.6060]
(c) Microsoft Corporation. All rights reserved.

C:\Users\arvin>jshell
|  Welcome to JShell -- Version 25.0.1
|  For an introduction type: /help intro

jshell> String s = "alok singh"
s ==> "alok singh"

jshell> s.length()
$2 ==> 10

jshell> s.
charAt(                 chars()                 codePointAt(            codePointBefore(        codePointCount(
codePoints()            compareTo(              compareToIgnoreCase(    concat(                 contains(
contentEquals(          describeConstable()     endsWith(               equals(                 equalsIgnoreCase(
formatted(              getBytes(               getChars(               getClass()              hashCode()
indent(                 indexOf(                intern()                isBlank()               isEmpty()
lastIndexOf(            length()                lines()                 matches(                notify()
notifyAll()             offsetByCodePoints(     regionMatches(          repeat(                 replace(
replaceAll(             replaceFirst(           resolveConstantDesc(    split(                  splitWithDelimiters(
startsWith(             strip()                 stripIndent()           stripLeading()          stripTrailing()
subSequence(            substring(              toCharArray()           toLowerCase(            toString()
toUpperCase(            transform(              translateEscapes()      trim()                  wait(
jshell> s.charAt(2)
$3 ==> 'o'

jshell> s.toLowerCase()
$4 ==> "alok singh"

jshell> s.toUpperCase()
$5 ==> "ALOK SINGH"

jshell> s.toCharArray()
$6 ==> char[10] { 'a', 'l', 'o', 'k', ' ', 's', 'i', 'n', 'g', 'h' }

jshell> s.startsWith(A)
|  Error:
|  cannot find symbol
|    symbol:   variable A
|  s.startsWith(A)
|               ^

jshell> s.startsWith('A')
|  Error:
|  incompatible types: char cannot be converted to java.lang.String
```

Command Prompt - jshell

```
jshell> s.startsWith('A')
|  Error:
|  incompatible types: char cannot be converted to java.lang.String
|  s.startsWith('A')
|               ^-^

jshell> s.toString(
Signatures:
String String.toString()

<press tab again to see documentation>
jshell> s.toString(
String String.toString()
This object (which is already a string!) is itself returned.

Returns:
the string itself.

<press tab again to see all possible completions; total possible completions: 1,413>
jshell> s.toString()
$7 ==> "alok singh"

jshell> s.indexOf('n')
$8 ==> 7

jshell> s.getBytes()
$9 ==> byte[10] { 97, 108, 111, 107, 32, 115, 105, 110, 103, 104 }

jshell> s.replace('n','m')
$10 ==> "alok simgh"

jshell> s.length()
$11 ==> 10

jshell> s.isEmpty()
$12 ==> false

jshell> s.compareTo("Alok")
$13 ==> 32

jshell> s.compareTo(
$10   $4    $5    $7    s

Signatures:
int String.compareTo(String anotherString)
```

Command Prompt - jshell

```
<press tab again to see documentation>
jshell> s.compareTo(
int String.compareTo(String anotherString)
Compares two strings lexicographically.The comparison is based on the Unicode value of each
character in the strings. The character sequence represented by this String object is compared
lexicographically to the character sequence represented by the argument string. The result is a
negative integer if this String object lexicographically precedes the argument string. The
result is a positive integer if this String object lexicographically follows the argument
string. The result is zero if the strings are equal; compareTo returns 0 exactly when the
#equals(Object) method would return true .
This is the definition of lexicographic ordering. If two strings are different, then either
they have different characters at some index that is a valid index for both strings, or their
lengths are different, or both. If they have different characters at one or more index
positions, let k be the smallest such index; then the string whose character at position k has
the smaller value, as determined by using the < operator, lexicographically precedes the other
string. In this case, compareTo returns the difference of the two character values at position
k in the two string -- that is, the value:
     this.charAt(k)-anotherString.charAt(k)

If there is no index position at which they differ, then the shorter string lexicographically
precedes the longer string. In this case, compareTo returns the difference of the lengths of
the strings -- that is, the value:
     this.length()-anotherString.length()

For finer-grained String comparison, refer to java.text.Collator .

Parameters:
anotherString - the String to be compared.

<press tab again to see next page>
jshell> s.replaceFirst('a','l')
|  Error:
|  incompatible types: char cannot be converted to java.lang.String
|  s.replaceFirst('a','l')
|                 ^-^

jshell> s.trim()
$14 ==> "alok singh"

jshell> s.intern(
Signatures:
String String.intern()

<press tab again to see documentation>
jshell> s.intern(
String String.intern()
```

Command Prompt - jshell

```
<press tab again to see documentation>
jshell> s.intern(
String String.intern()
Returns a canonical representation for the string object.
A pool of strings, initially empty, is maintained privately by the class String .
When the intern method is invoked, if the pool already contains a string equal to this String
object as determined by the #equals(Object) method, then the string from the pool is returned.
Otherwise, this String object is added to the pool and a reference to this String object is
returned.
It follows that for any two strings s and t , s.intern() == t.intern() is true if and only if
s.equals(t) is true .
All literal strings and string-valued constant expressions are interned. String literals are
defined in section 3.10.5 of the The Java Language Specification .

Returns:
a string that has the same contents as this string, but is guaranteed to be from a pool of
unique strings.

<press tab again to see all possible completions; total possible completions: 1,421>
jshell> s.intern()
$15 ==> "alok singh"

jshell> s.codePoints()
$16 ==> java.util.stream.IntPipeline$Head@41cf53f9

jshell> s.startsWith('a")
|  Error:
|  unclosed character literal
|  s.startsWith('a")
|               ^

jshell> s.startsWith('a')
|  Error:
|  incompatible types: char cannot be converted to java.lang.String
|  s.startsWith('a')
|               ^-^

jshell> s.startsWith("A")
$17 ==> false

jshell> s.concat("A")
$18 ==> "alok singhA"

jshell> s.concat("IT")
$19 ==> "alok singhIT"
```

Command Prompt - jshell

```
jshell> s.replaceFirst(['a','l'])
|  Error:
|  illegal start of expression
|  s.replaceFirst(['a','l'])
|                 ^
|  Error:
|  ']' expected
|  s.replaceFirst(['a','l'])
|                      ^

jshell> s.indexOf("Singh")
$20 ==> -1

jshell> s.indexOf("singh")
$21 ==> 5

jshell> s.lastIndexOf("h")
$22 ==> 9

jshell> s.equals("alok singh")
$23 ==> true

jshell> s.equalsIgnoreCase("aloksingh")
$24 ==> false

jshell> s.compare("alok")
|  Error:
|  cannot find symbol
|    symbol:   method compare(java.lang.String)
|  s.compare("alok")
|  ^-------^

jshell> s.compareTo("Alok")
$25 ==> 32

jshell> String name = "   alok  Singh  "
name ==> "   alok  Singh  "

jshell> name.trim()
$27 ==> "alok  Singh"

jshell> s.contains("al")
$28 ==> true
```

```
jshell> name.trim()
$27 ==> "alok  Singh"

jshell> s.contains("al")
$28 ==> true


jshell> s.split()
|  Error:
|  no suitable method found for split(no arguments)
|      method java.lang.String.split(java.lang.String,int) is not applicable
|        (actual and formal argument lists differ in length)
|      method java.lang.String.split(java.lang.String,int,boolean) is not applicable
|        (actual and formal argument lists differ in length)
|      method java.lang.String.split(char,int,boolean) is not applicable
|        (actual and formal argument lists differ in length)
|      method java.lang.String.split(java.lang.String) is not applicable
|        (actual and formal argument lists differ in length)
|  s.split()
|  ^-----^

jshell> s.toUpperCase()
$29 ==> "ALOK SINGH"

jshell> /exit
|  Goodbye
```